

Reverse Engineering Write-up – razkom_v1

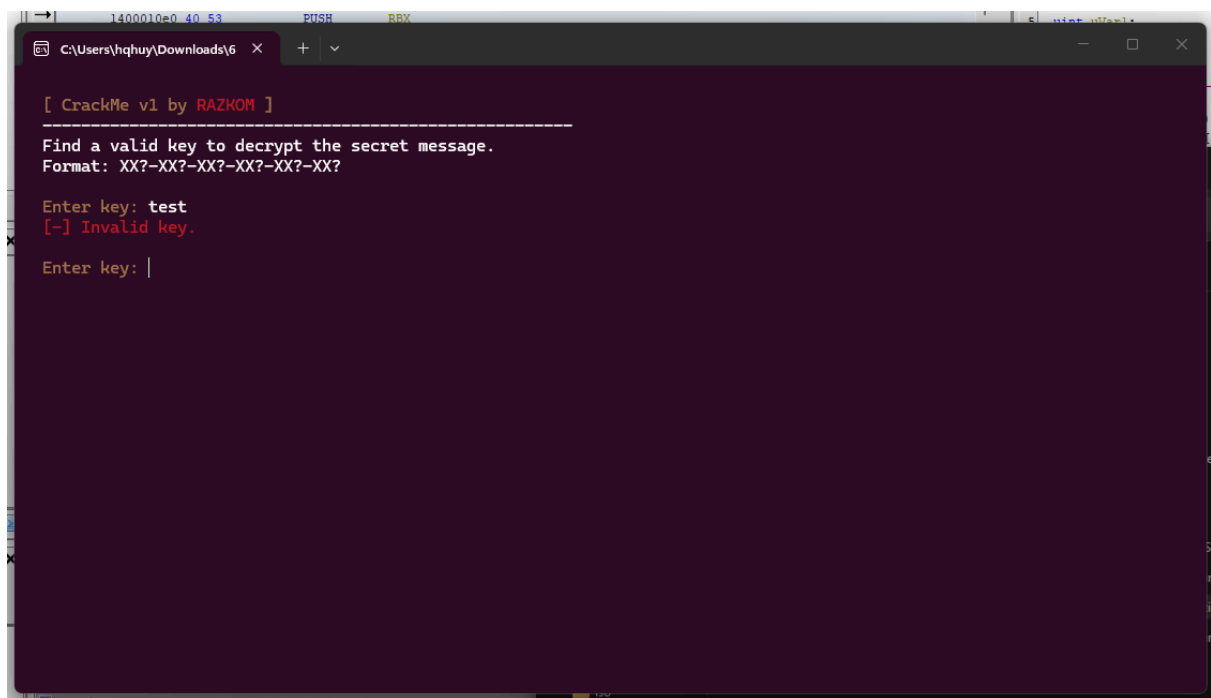
Overview

This write-up describes how the key-checking logic of razkom_v1.exe was reversed using Ghidra in order to generate a valid key that matches the required format:

Format: XX?-XX?-XX?-XX?-XX?-XX?

Step 1 – Run the program and observe behavior

First, razkom_v1.exe was executed to understand what the program expects. The program displays a banner and prompts the user to enter a key. Testing with a simple input such as "test" results in "Invalid key", confirming that the input is validated by an internal routine.



```
1400010e0 40 53 PUSH RBX
C:\Users\hghuy\Downloads\6 x + v - _ x
[ CrackMe v1 by RAZKOM ]
-----
Find a valid key to decrypt the secret message.
Format: XX?-XX?-XX?-XX?-XX?-XX?

Enter key: test
[~] Invalid key.

Enter key: |
```

Step 2 – Identify the key validation function in Ghidra

The binary was opened in Ghidra and analyzed. In the main logic, the local variable local_68 is assigned the address of a function named FUN_1400010e0.

This function is used to check whether the entered key is valid. Navigating into FUN_1400010e0 reveals the core validation logic.

Step 3 – Analyze FUN_1400010e0 and extract constraints

The decompiled function is:

```
ulonglong FUN_1400010e0(char *param_1)

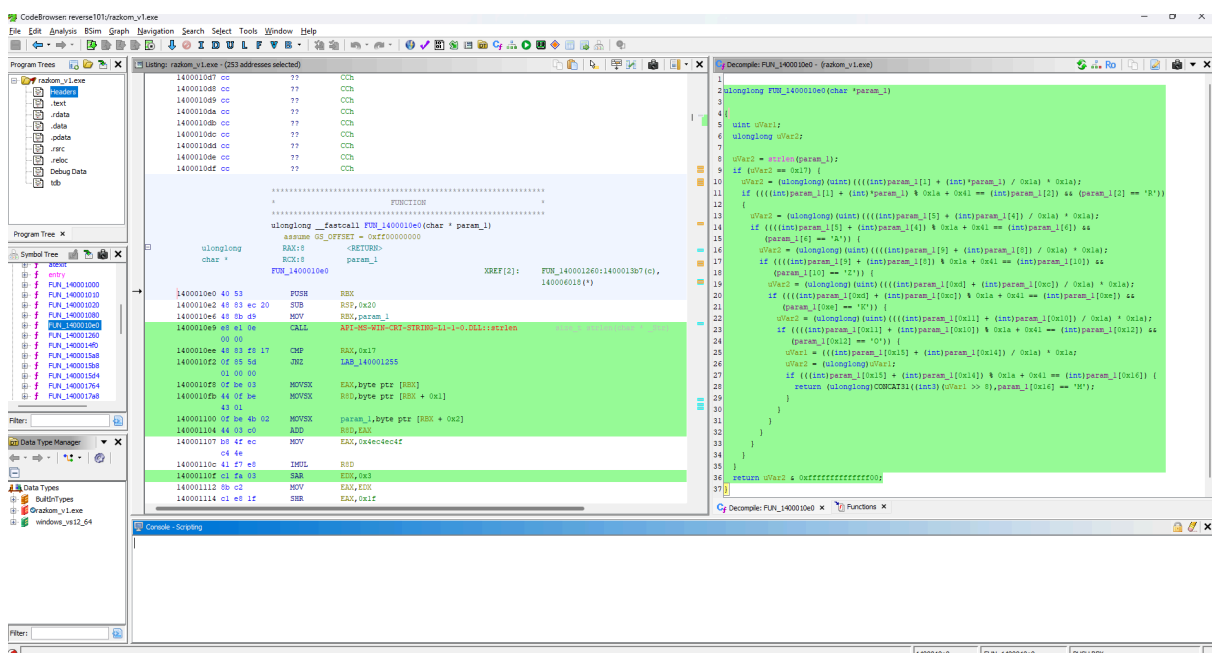
{
    uint uVar1;
    ulonglong uVar2;

    uVar2 = strlen(param_1);
    if (uVar2 == 0x17) {
        uVar2 = (ulonglong)(uint)((((int)param_1[1] + (int)*param_1) / 0x1a) * 0x1a);
        if (((((int)param_1[1] + (int)*param_1) % 0x1a + 0x41 == (int)param_1[2])) && (param_1[2] == 'R'))
        {
            uVar2 = (ulonglong)(uint)((((int)param_1[5] + (int)param_1[4]) / 0x1a) * 0x1a);
            if (((((int)param_1[5] + (int)param_1[4]) % 0x1a + 0x41 == (int)param_1[6])) &&
                (param_1[6] == 'A')) {
                uVar2 = (ulonglong)(uint)((((int)param_1[9] + (int)param_1[8]) / 0x1a) * 0x1a);
                if (((((int)param_1[9] + (int)param_1[8]) % 0x1a + 0x41 == (int)param_1[10])) &&
                    (param_1[10] == 'Z')) {
                    uVar2 = (ulonglong)(uint)((((int)param_1[0xd] + (int)param_1[0xc]) / 0x1a) * 0x1a);
                    if (((((int)param_1[0xd] + (int)param_1[0xc]) % 0x1a + 0x41 == (int)param_1[0xe])) &&
                        (param_1[0xe] == 'K')) {
                        uVar2 = (ulonglong)(uint)((((int)param_1[0x11] + (int)param_1[0x10]) / 0x1a) * 0x1a);
                        if (((((int)param_1[0x11] + (int)param_1[0x10]) % 0x1a + 0x41 == (int)param_1[0x12])) &&
```

```

        (param_1[0x12] == '0')) {
            uVar1 = (((int)param_1[0x15] + (int)param_1[0
x14]) / 0x1a) * 0x1a;
            uVar2 = (ulonglong)uVar1;
            if (((int)param_1[0x15] + (int)param_1[0x14])
% 0x1a + 0x41 == (int)param_1[0x16]) {
                return (ulonglong)CONCAT31((int3)(uVar1 >>
8),param_1[0x16] == 'M');
            }
        }
    }
}
}
}
}
return uVar2 & 0xffffffffffff00;

```



Key observations

1. Length constraint

The function requires:

```
strlen(param_1) == 0x17
```

0x17 = 23 characters. This matches the expected pattern of 6 groups separated by 5 dashes:

$$(3 * 6) + 5 = 18 + 5 = 23$$

So the key format is:

XXX-XXX-XXX-XXX-XXX-XXX

1. Character equation

Each 3-character group is validated with the same pattern:

$$((\text{param}[\text{i}] + \text{param}[\text{i}+1]) \bmod 26) + 'A' == \text{param}[\text{i}+2]$$

Where:

- 0x1a = 26
- 0x41 is ASCII 'A'

This is equivalent to a modular arithmetic mapping to uppercase letters.

1. Fixed characters

In addition to the arithmetic check, the third character of each group is forced to a fixed uppercase letter:

```
param_1[2] == 'R'
param_1[6] == 'A'
param_1[10] == 'Z'
param_1[14] == 'K'
param_1[18] == 'O'
param_1[22] == 'M'
```

Therefore, the key must embed the string:

R A Z K O M

Step 4 – Convert each check into constraints per block

Define:

$$f(a, b) = ((a + b) \bmod 26) + 'A'$$

Then the function requires the following for each block:

```
param_1[2] == 'R' and param_1[2] == f(param_1[0], param_1[1])
param_1[6] == 'A' and param_1[6] == f(param_1[4], param_1[5])
param_1[10] == 'Z' and param_1[10] == f(param_1[8], param_1[9])
param_1[14] == 'K' and param_1[14] == f(param_1[12], param_1[13])
param_1[18] == '0' and param_1[18] == f(param_1[16], param_1[17])
param_1[22] == 'M' and param_1[22] == f(param_1[20], param_1[21])
```

Because the program does not restrict param_1[0] and param_1[1] (and similarly for other blocks) beyond this formula, there are many valid solutions. One easy approach is to choose a constant middle character for each block (for example 'A') and solve for the first character.

Example for the first block (positions 0,1,2)

We want:

```
((param[0] + param[1]) mod 26) + 'A' = 'R'
```

Pick param[1] = 'A' (ASCII 65). If we choose param[0] = 'R' (ASCII 82):

```
82 + 65 = 147
147 mod 26 = 17
17 + 65 = 82 -> 'R'
```

So "RAR" satisfies the first block.

Step 5 – Solve the full key with a Python script

A short script can generate valid blocks for each target letter in "RAZKOM" using a fixed middle character 'A':

```
import string
```

```

ALPH = string.ascii_uppercase

def solve_block_with_fixed_middle(target_char: str, middle
='A'):
    for x in ALPH:
        computed = chr(((ord(x) + ord(middle)) % 26) + ord
('A'))
        if computed == target_char:
            return x + middle + target_char
    raise ValueError(f"No solution for {target_char} with m
iddle={middle}")

def build_key():
    targets = "RAZKOM"
    blocks = [solve_block_with_fixed_middle(t, middle='A')
for t in targets]
    return "-".join(blocks)

if __name__ == "__main__":
    print("Solved key:", build_key())

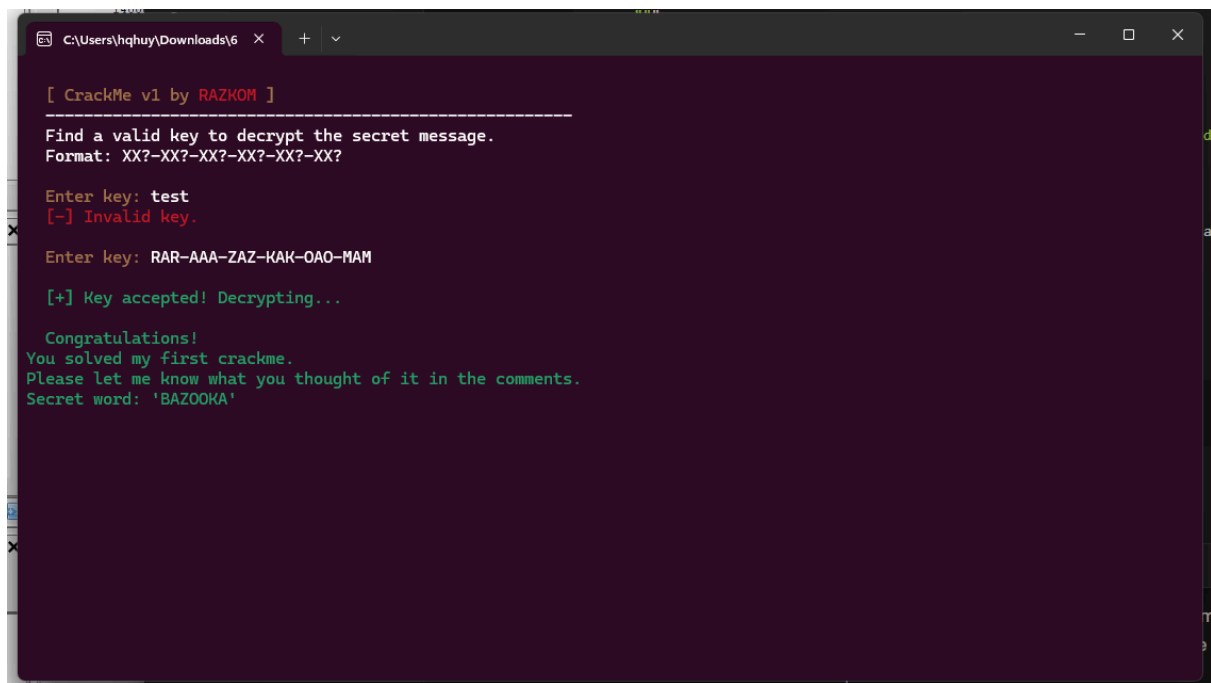
```

Running the script produces:

```
Solved key: RAR-AAA-ZAZ-KAK-OAO-MAM
```

Result

Entering the generated key RAR-AAA-ZAZ-KAK-OAO-MAM satisfies the arithmetic constraints and the fixed-character checks enforced by FUN_1400010e0. This key matches the required format XXX-XXX-XXX-XXX-XXX-XXX and passes validation.



```
[ CrackMe v1 by RAZKOM ]
-----
Find a valid key to decrypt the secret message.
Format: XX?-XX?-XX?-XX?-XX?-XX?

Enter key: test
[-] Invalid key.

Enter key: RAR-AAA-ZAZ-KAK-OAO-MAM

[+] Key accepted! Decrypting...

Congratulations!
You solved my first crackme.
Please let me know what you thought of it in the comments.
Secret word: 'BAZOOKA'
```

Conclusion

Using Ghidra, the key validation routine FUN_1400010e0 was located and reversed. The function enforces a length requirement (23 characters) and validates six 3-character blocks separated by dashes. Each block uses modular arithmetic over 26 letters and forces the third character to spell "RAZKOM". By translating the checks into equations and solving them with a simple Python script, a valid key was generated.